Отчёт по практике 5

Использование статического анализатора кода для обеспечения безопасности

Выполнил: Колесников Андрей Юрьевич, 4 курс, группа БСБО-03-23, студенческий билет 23Б0099, номер по журналу 10

РТУ МИРЭА, Институт кибербезопасности и цифровых технологий

1. Цель работы

Нужно было найти статическим анализатором участок с SQL-инъекцией в модуле Blind SQL Injection учебного стенда DVWA, разобрать этот код и указать слабые места, а затем сделать свою страницу вывода информации об объекте, которая не позволяет внедрить SQL. Дополнительно требовалось подтвердить уязвимость практическими инструментами — sqlmap и Burp — и показать, что защищённая страница сохраняет формат GET-запроса из задания, но остаётся безопасной.

2. Стенд

DVWA версии 1.10 поднят в Docker на 127.0.0.1:8081. Использован тот же образ vulnerables/web-dvwa, что и в практике 4; база инициализирована через setup.php, уровень защиты выставлен low. Наружу стенд не публиковался: порт привязан к петлевому адресу. Все запросы sqlmap, Burp и собственных скриптов шли только на этот локальный адрес.

Статический анализ выполнялся двумя инструментами. Основной — Semgrep в контейнере semgrep/semgrep с набором правил p/php. Дополнительно развёрнут SonarQube Community 26.10 в контейнере, проект dvwa-p5 просканирован sonar-scanner. Для практической проверки использованы sqlmap (образ parrotsec/sqlmap) и Burp Suite Community 2026.3.3, запущенный под Xvfb в контейнере с прокси на 127.0.0.1:8085.

Компонент	Версия / адрес	Назначение
DVWA	1.10, http://127.0.0.1:8081	уязвимый стенд
Semgrep	образ semgrep/semgrep, правила p/php	поиск SQL-инъекции
SonarQube	Community 26.10, http://127.0.0.1:9000	дополнительный SAST
sqlmap	parrotsec/sqlmap	подтверждение и эксплуатация
Burp Suite	Community 2026.3.3, proxy 127.0.0.1:8085	перехват запросов
Своя страница	PHP 8.3, PDO/SQLite, http://127.0.0.1:8082	защищённый вывод объекта

3. Статический анализ

Исходники модулей sqli и sqli_blind скопированы из контейнера DVWA в каталог vulnerable/dvwa. Semgrep запускался командой:

docker run --rm -v "$PWD/vulnerable:/src" -w /src semgrep/semgrep semgrep scan --config p/php .

Найдено два срабатывания уровня ERROR, оба — правило php.lang.security.injection.tainted-sql-string.tainted-sql-string, CWE-89. Одно указывает на vulnerable/dvwa/sqli/low.php:8, второе — на vulnerable/dvwa/sqli_blind/low.php:8. В обоих случаях пользовательский ввод попадает в строку SQL, которая затем передаётся в mysqli_query().

Файл	Строка	Правило	CWE	Уровень
vulnerable/dvwa/sqli/low.php	8	tainted-sql-string	CWE-89	ERROR
vulnerable/dvwa/sqli_blind/low.php	8	tainted-sql-string	CWE-89	ERROR

SonarQube просканировал те же три файла (40 строк кода) профилем Sonar way comprehensive, но выдал 0 issues. Это не значит, что код чистый: в бесплатной сборке SonarQube для PHP нет полноценного taint-анализа, а правило php:S2077 ловит не каждую конкатенацию. Поэтому основной результат в работе — находка Semgrep, а SonarQube использован как дополнительный контроль и источник метрик. Этот момент важно проговорить на защите: инструмент не «доказывает» отсутствие уязвимости, если у него нет нужного типа анализа.

4. Код-ревью

В модуле low.php значение берётся из $_REQUEST['id'] и без проверки подставляется в запрос:

$id = $_REQUEST['id']; $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';"; $result = mysqli_query($GLOBALS["___mysqli_ston"], $query);

Это классическая CWE-89. $_REQUEST принимает и GET, и POST, и cookie, поэтому способ передачи не важен. Кавычка в id закрывает строковый литерал, после чего можно дописать UNION SELECT, логическое условие или подзапрос. В blind-варианте результат не выводится целиком, но разница между «User ID exists» и «User ID is MISSING» даёт булев канал: по нему sqlmap и восстанавливает данные по битам. В sqli/low.php результат выводится напрямую, поэтому там возможен UNION и чтение произвольных таблиц, включая users с хэшами паролей.

В medium.php добавлен mysqli_real_escape_string, а запрос стал числовым: WHERE user_id = $id без кавычек. Экранирование здесь не помогает, потому что оно рассчитано на

строковый литерал; в числовом контексте ввод 1 OR 1=1 остаётся синтаксически допустимым. Это хороший пример того, почему экранирование нельзя считать заменой параметризации. Правильное решение — подготовленный запрос с bind_param('i') или PDO с параметром :id, где тип проверяется отдельно.

Слабость	Где	CWE	Последствие
Конкатенация ввода в SQL	low.php, sqli_blind/low.php	CWE-89	чтение и изменение данных, обход логики
Экранирование в числовом контексте	medium.php	CWE-89	инъекция вида 1 OR 1=1
Вывод ошибок СУБД	medium.php, mysqli_error()	CWE-209	раскрытие структуры БД
Нет ограничения запросов	все уровни	CWE-770	автоматизированный перебор условий
Вывод id без экранирования	sqli/low.php	CWE-79	отражённый XSS через параметр id

Исправление исходного участка: заменить mysqli_query с конкатенацией на mysqli_prepare + bind_param, либо перейти на PDO с PDO::ATTR_EMULATE_PREPARES = false. Идентификаторы таблиц и полей, если они когда-нибудь понадобятся из ввода, выбирать только из белого списка. Ошибки СУБД писать в лог, а пользователю показывать общее сообщение.

5. Проверка sqlmap

sqlmap запускался по blind-странице с cookie авторизованной сессии DVWA:

```
docker run --rm --add-host=host.docker.internal:host-gateway parrotsec/sqlmap -u "http://host.docker.internal:8081/vulnerabilities/sqli_blind/?id=1&Submit=Submit" --cookie="PHPSESSID=...; security=low" -p id --batch --dbms=MySQL --technique=B --current-db --dbs
```

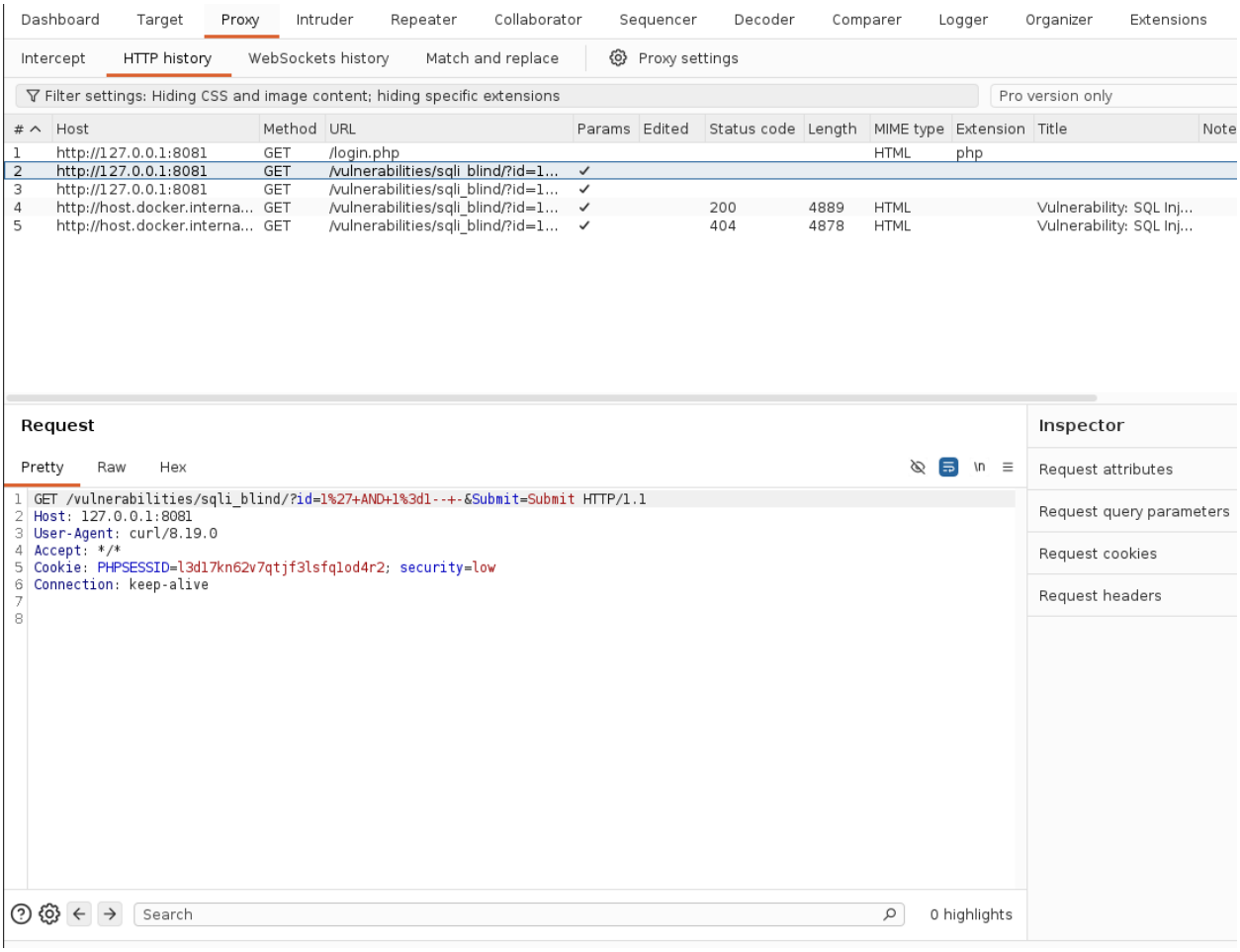
Инструмент определил параметр id как инъективный, тип — boolean-based blind, СУБД — MySQL/MariaDB. Current database: dvwa. Доступные базы: dvwa и information_schema. Дополнительно выполнен дамп одной строки users:

Поле	Значение
user_id	1
user	admin
password	5f4dcc3b5aa765d61d8327deb882cf99 (MD5 от password)
first_name / last_name	admin / admin
last_login	2026-10-24 00:43:56

Дамп показывает практический эффект: через blind-инъекцию можно вытащить учётные данные администратора. Пароль здесь учебный и совпадает с паролем стенда по умолчанию; на реальной системе дальше была бы попытка подбора хэша или повторного использования пароля. Все действия выполнялись на локальном контейнере, чужие системы не затрагивались.

6. Проверка через Burp Suite

Burp Community запущен в контейнере под Xvfb, прокси слушает 127.0.0.1:8085. Через него отправлены два запроса к blind-странице: с условием 1=1 и с условием 1=2. В HTTP history видно, что первый вернул 200 и текст «User ID exists in the database», второй — 404 и «User ID is MISSING from the database». Этого различия достаточно, чтобы вручную подтвердить булевую инъекцию и повторить её в Repeater.



На скриншоте выбран запрос с payload id=1' AND 1=1-- -; в нижней панели виден сам запрос, cookie сессии и параметр id. Заголовки ответа и тело сохранены в каталоге burp и results, поэтому результат можно проверить без повторного запуска Burp.

7. Защищённая страница вывода объекта

Страница object_info.php сделана на PHP 8.3 с PDO/SQLite. Формат запроса сохранён как в задании: username, password, user_token, Login и id передаются методом

GET. После успешного входа пароль убирается из адресной строки редиректом на object_info.php?id=N, а сессия хранится в cookie.

Ключевые отличия от DVWA: логин и пароль проверяются подготовленным запросом; пароль хранится как password_hash(), а не MD5; id приводится к целому через FILTER_VALIDATE_INT и всё равно передаётся в SQL отдельным параметром :id; CSRF-токен обязателен; после пяти неудач срабатывает блокировка; вывод экранируется htmlspecialchars(). Даже строка 2' OR '1'='1 в id не меняет запрос — она либо отбрасывается валидацией, либо воспринимается как значение.

Проверки страницы:

Проверка	Результат
Вход с верными данными	302 и переход на object_info.php?id=2
Вывод объекта 2	200, отображаются «Счёт за сентябрь» и petrov
SQL-инъекция в логине ' OR '1'='1	вход отклонён, объект не показан
SQL-инъекция в id 2' OR '1'='1	показан только объект 1, чужие не выводятся
Запрос без user_token	вход запрещён, выведено сообщение об устаревшей сессии

Тест запускается командой python tests/test_object_info.py при поднятом контейнере p5-secure. Все пять проверок пройдены, код возврата 0.

7.1. Соответствие требованиям

Пункт задания	Что сделано	Где смотреть
1. Найти SQL-инъекцию статическим анализатором	Semgrep, правила p/php; SonarQube как дополнительный контроль	раздел 3, semgrep_summary.txt
2. Проанализировать код и сделать кодревью	разобраны low, medium, blind; указана CWE-89	раздел 4, таблица слабостей
3. Своя система вывода объекта без SQL-инъекции	object_info.php на PDO, параметры, CSRF, rate limit	раздел 7, SecureObjectInfo
4. Использовать sqlmap	boolean-based blind, current db, дамп admin	раздел 5, sqlmap_blind.log и sqlmap_dump.log
5. Использовать Burp	запросы через proxy 8085, разница 200/404	раздел 6, burp_history_selected_cropped.png
GET-формат как в задании	username, password, user_token, Login, id	object_info.php?username=...&password=...&user_token=...&Login=Login&id=2

8. Выводы

Статический анализ нашёл SQL-инъекцию в двух местах DVWA и показал, что проблема находится именно в конкатенации пользовательского ввода с текстом запроса.

sqlmap подтвердил эксплуатабельность blind-варианта и вытащил учётную запись администратора; Burp позволил увидеть разницу ответов 200/404 на булевы условия. Это не лабораторная «на бумаге», а воспроизводимый результат.

Защищённая страница сохраняет требуемый GET-формат, но использует параметризованные запросы, нормальное хэширование, CSRF-токен и ограничение попыток. Остаточный риск — сам GET-формат: пароль попадает в журналы и историю браузера. Это требование задания, поэтому риск вынесен в отчёт явно; в реальной системе форму входа нужно переводить на POST.

Ограничение работы: SonarQube Community не выполнил taint-анализ PHP, поэтому его нулевой результат не считается доказательством отсутствия уязвимости. Основной вывод сделан по Semgrep и подтверждён sqlmap/Burp.